

SYMBOLICAI: A FRAMEWORK FOR LOGIC-BASED APPROACHES COMBINING GENERATIVE MODELS AND SOLVERS

Marius-Constantin Dinu^{* † ‡} Claudiu Leoveanu-Condrei^{† ‖} Markus Holzleitner[‡]
 Werner Zellinger^{‡ §} Sepp Hochreiter[‡]
 ExtensityAI[†] Johannes Kepler University[‡] RICAM[§] Amazon Devices[‖]

ABSTRACT

We introduce *SymbolicAI*, a versatile and modular framework employing a logic-based approach to concept learning and flow management in generative processes. SymbolicAI enables the seamless integration of generative models with a diverse range of solvers by treating large language models (LLMs) as semantic parsers that execute tasks based on both natural and formal language instructions, thus bridging the gap between symbolic reasoning and generative AI. We leverage probabilistic programming principles to tackle complex tasks, and utilize differentiable and classical programming paradigms with their respective strengths. The framework introduces a set of polymorphic, compositional, and self-referential operations for multi-modal data that connects multi-step generative processes and aligns their outputs with user objectives in complex workflows. As a result, we can transition between the capabilities of various foundation models with in-context learning capabilities and specialized, fine-tuned models or solvers proficient in addressing specific problems. Through these operations based on in-context learning our framework enables the creation and evaluation of explainable computational graphs. Finally, we introduce a quality measure and its empirical score for evaluating these computational graphs, and propose a benchmark that compares various state-of-the-art LLMs across a set of complex workflows. We refer to the empirical score as the "Vector Embedding for Relational Trajectory Evaluation through Cross-similarity", or *VERTEX* score for short. The [framework codebase](#)¹ and [benchmark](#)² are linked below.

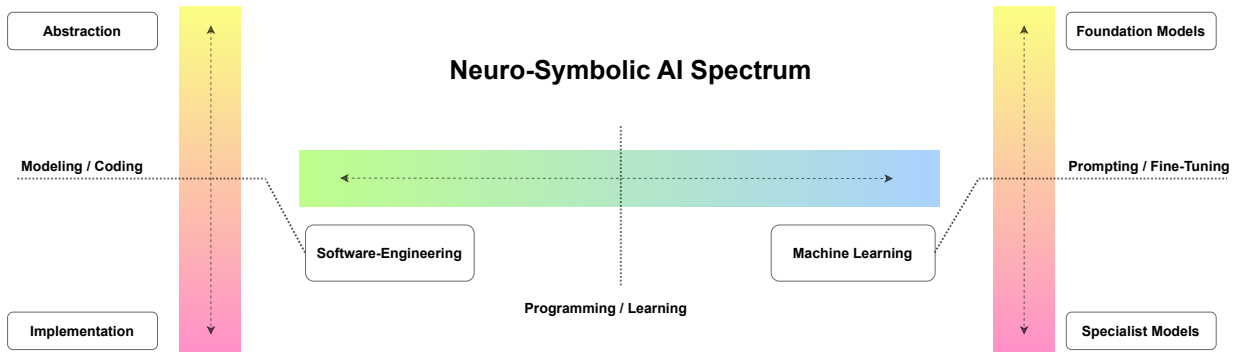


Figure 1: Our neuro-symbolic framework enables a seamless transition between symbolic and differentiable programming, each with distinct dynamics and strengths. Differentiable programming provides access to foundational and specialist models. Classical programming, on the other hand, shifts between abstraction and implementation, focusing on high-level concepts before delving into the details of implementation.

¹ SymbolicAI framework: <https://github.com/ExtensityAI/symbolicai>

² Evaluation benchmark: <https://github.com/ExtensityAI/benchmark>

* Correspondence to: dinu@ml.jku.at, {marcus, leo}@extensity.ai

‖ Work done outside of Amazon.

1 INTRODUCTION

The recent surge in generative AI, particularly involving large language models (LLMs), has demonstrated their wide-ranging applicability across various domains (Badita, 2022; Degraeve, 2022). These models have enhanced the functionality of tools for search-based interactions (YouWrite, 2022; Writesonic, 2022; Microsoft, 2023), program synthesis (Jain et al., 2021; Romera-Paredes et al., 2023; Key et al., 2023), chat-based interactions (ReplikaAI, 2016; OpenAI, 2022; Google, 2023), and many more. Moreover, language-based approaches have facilitated connections between different modalities, enabling text-to-image (Ramesh et al., 2021; Saharia et al., 2022), text-to-video (Singer et al., 2022), text-to-3D (Poole et al., 2022), text-to-audio (Oord et al., 2016; Wang et al., 2017), and text-to-code (Wang et al., 2021b; Lu et al., 2021; Li et al., 2022b) transformations, to name a few. Consequently, by training on vast quantities of unlabelled textual data, LLMs have been shown to not only store factual knowledge (Petroni et al., 2019; Kassner et al., 2020) and approximate users’ intentions to some extent (Andreas, 2022), but also to unlock deep specialist capabilities through innovative prompting techniques (Nori et al., 2023).

Despite their versatility, current LLMs face challenges such as fallacious reasoning and the generation of erroneous content, commonly referred to as hallucinations (Jones & Steinhardt, 2022). These limitations highlight the importance of integrating complementary symbolic methods to validate and guide the generative processes of LLMs, ensuring more accurate and reliable outputs. In parallel, efforts have focused on developing tool-based approaches (Schick et al., 2023) or template frameworks (Chase, 2023) to extend LLMs’ capabilities and enable a broader spectrum of applications. However, these efforts only partially capture the potential inherent in leveraging LLMs as *semantic parsers*. In contrast to parsers for structured languages a semantic parser is able to break down unstructured human language into semantically meaningful components and transform those into a structured form. While traditionally semantic parsing has been a role filled by specialized algorithms and models, we posit that LLMs, through their training on diverse linguistic data, have developed the ability to perform semantic parsing as part of their broader natural language processing capabilities. In turn, we identify LLMs as a central component in creating sophisticated neuro-symbolic (NeSy) AI systems. These systems integrate symbolic and sub-symbolic concepts and utilize the capabilities of semantic parsing to develop symbolic expressions that enable new probabilistic programming paradigms.

We introduce *SymbolicAI*, a compositional NeSy framework able to represent and manipulate multi-modal and self-referential structures (Schmidhuber, 2007; Fernando et al., 2023). SymbolicAI augments the generative process of LLMs with in-context learning operations, realized through functional primitives, and enables the creation of versatile applications through in-context learning (Wei et al., 2022a). These operations enable logic-based components that guide the generative process and enable a modular NeSy system, including a wide range of existing solvers, formal language engines for mathematical expression evaluation, theorem provers, knowledge bases, and search engines for information retrieval. SymbolicAI exposes these solvers as building blocks for constructing compositional functions as computational graphs, making it possible to bridge classical and differentiable programming paradigms with the aim to create *domain-invariant problem solvers*. In designing the architecture of SymbolicAI, we drew inspiration from a body of evidence that suggests the human brain possesses a selective language processing module (Macswiney, 2002; Fedorenko et al., 2010; Menenti et al., 2011; Regev et al., 2013; Scott et al., 2016; Deniz et al., 2019; Hu et al., 2022), prior research on cognitive architectures (Newell & Simon, 1956; Newell et al., 1957; Newell & Simon, 1972; Newell, 1990; Laird, 2022), and the significance of language on the structure of semantic maps in the human brain (Huth et al., 2016). We consider language as a central processing module, distinct from other cognitive processes such as reasoning or memory (Paischer et al., 2022; 2023). We hypothesize that such a central processing module based in language is a core component of broad AI systems (see Appendix Section A) and enables the development of fully autonomous AI systems for decision-making.

A significant challenge encountered in the development of our framework pertained to the evaluation of LLMs when used as semantic parsers in a NeSy workflow. Current evaluation of generated content relies on metrics for single-step generative processes, such as the BLEU score (Papineni et al., 2002). These metrics are not suitable for evaluating multi-step generative processes. BLEU has limitations, as it measures n -gram-based overlap of generated output with a reference that does not consider the semantic meaning. As a result, BLEU fails to capture semantic equivalence, especially in more complex tasks. More recent metrics such as CIDEr (Vedantam et al., 2014) or SPICE (Anderson et al., 2016) are also not suitable for our problem, either because they are built upon BLEU (in case of CIDEr) or designed with inductive biases specifically for image captioning.

Therefore, alongside our framework we introduce a quality measure (and its empirical score) for multi-step generative processes based on semantic meaning. We refer to our score as "Vector Embedding for Relational Trajectory Evaluation through Cross-similarity", or VERTEX score for short. Our VERTEX score uses embeddings to compare node distributions within a computational graph. It measures the semantic meaning across the distributional path by computing at each node the cross-similarity between the generated embeddings and embeddings sampled from a reference

distribution. Furthermore, the VERTEX score is designed such that it can be used as a reward signal in a reinforcement learning setting (Sutton, 1984). Finally, we propose a benchmark for evaluating complex workflows. We define a set of basic evaluations, particularly associative predictions based on in-context learning, multi-modal bindings for tool utilization, and program synthesis for subroutine execution. Furthermore, we introduce complex evaluations for logic-based components and hierarchical computational graphs.

In summary, the key contributions presented in this work are as follows:

- We introduce SymbolicAI, a logic-based framework for concept learning and flow management in generative processes, enabling seamless integration with a wide range of foundation models and solvers.
- We leverage LLMs as semantic parsers to enable the creation of complex computational graphs by combining symbolic expressions with probabilistic programming paradigms.
- We introduce a quality measure and its empirical score alongside a benchmark designed for multi-step generative processes for comparing LLMs across a wide range of complex tasks.

2 RELATED WORK

Symbolic Methods The field of symbolic AI has its foundations in the works of the Logic Theorist (LT) (Newell & Simon, 1956) and the General Problem Solver (GPS) (Newell et al., 1957). These programs represented the first steps towards automated reasoning and problem-solving utilizing symbolic representations. Despite their advancements, both faced challenges in dealing with the complexity of real-world problems, particularly due to the combinatorial nature of the solution space. To address these limitations, the Soar (Laird et al., 1987) cognitive architecture was developed, advancing the notion that intelligent behavior results from goal-oriented search through a problem space (Newell & Simon, 1972; McCarthy et al., 2006), with each step consisting of selecting and applying operators. Soar introduced components like reinforcement learning, impasses, sub-states, and chunking to enhance its problem-solving capabilities. It also demonstrated the importance of learning from experiences to adapt and improve performance over time. However, Santoro et al. (2022) emphasizes the subjectivity of symbols and suggests that human-like symbolic fluency could develop in machines through learning algorithms immersed in socio-cultural contexts. This perspective, anchored in the notion that symbols are triadic and their meaning emerges from consensus, seeks to move away from traditional symbolic AI methodologies towards AI that adaptively learns meaning and behaviors from human-like experiences. The goal is to cultivate machines that demonstrate symbolic behaviors across a spectrum of competencies, potentially mirroring the evolutionary and social learning processes observed in humans. Lastly, symbolic AI struggles with real-world data’s unpredictability and variability. These challenges have led to the employment of statistical learning methodologies, like deep learning (Alom et al., 2018), which are more adept at managing noise and uncertain information through vector-valued representations.

Sub-Symbolic Methods The sub-symbolic framework, rooted in neural network paradigms, began with pioneering works such as the perceptron (McCulloch & Pitts, 1943), with the first hardware implementation quickly following (Rosenblatt, 1958). The foundational notion of distributed processing (Rumelhart et al., 1986) was later bolstered and further expanded by demonstrating that multilayer feedforward networks with a single hidden layer can serve as universal approximators for any Borel measurable function, given sufficient hidden units (Hornik et al., 1989). Fast-forward, contemporary frameworks achieve a significant leap with the introduction of the Transformer architecture (Vaswani et al., 2017), which underpins most of today’s LLMs. These LLMs demonstrate exceptional capabilities in in-context learning, a method popularized by the likes of GPT-3 (Brown et al., 2020), where models improve task performance through natural language instruction and examples provided directly in the input prompt. While in-context learning bypasses the need for explicit retraining, it demands meticulous prompt design to steer models towards desired behaviors.

Neuro-Symbolic Methods To overcome the limitations of each individual method, NeSy approaches meld the statistical inference strengths of deep neural architectures with the generalization and explainability of symbolic systems (Garcez et al., 2015; Besold et al., 2017; d’Avila Garcez et al., 2019; d’Avila Garcez & Lamb, 2020; Lamb et al., 2020; Hamilton et al., 2022; Yu et al., 2023). Some approaches focus on different strategies for integrating learning and reasoning processes (Yu et al., 2023; Fang et al., 2024). Firstly, *learning for reasoning* methods treat the learning aspect as an accelerator for reasoning, in which deep neural networks are employed to reduce the search space for symbolic systems (Silver et al., 2016; 2017b;a; Qu & Tang, 2019; Schrittwieser et al., 2020). Secondly, *reasoning for learning* views reasoning as a way to regularize learning, in which symbolic knowledge acts as a guiding constraint that oversees machine learning tasks (Hu et al., 2016; Xu et al., 2018). Thirdly, the *learning-reasoning* category enables a symbiotic relationship between learning and reasoning. Here, both elements interact and share information to boost

problem-solving capabilities (Donadello et al., 2017; Manhaeve et al., 2018; Mao et al., 2019; Ellis, 2023). This synergy further extends when considering graph-based methods, which closely align with the objectives of our proposed framework. Research in this area, such as CycleGT (Guo et al., 2020) and Paper2vec (Ganguly & Pudi, 2017) explores unsupervised techniques for bridging graph and text representations, GPTSwarm (Zhuge et al., 2024) explores graph optimizers to refine node-level prompts and edge optimization. Subsequently, graph embeddings, when utilized within symbolic frameworks, can enhance knowledge graph reasoning tasks (Zhang et al., 2021), or more generally, provide the bedrock for learning domain-invariant representations (Park et al., 2023).

Lastly, building upon the insights from Sun et al. (2022), the integration of NeSy techniques in scientific workflows promises significant acceleration in scientific discovery. While previous work has effectively identified opportunities and challenges, we have taken a more ambitious approach by developing a comprehensive framework from the ground up to facilitate a wide range of NeSy integrations.

Large Language Models In part, instruction-based fine-tuning of LLMs through reinforcement learning from human feedback (Ouyang et al., 2022; Li et al., 2023) or direct preference optimization (Rafailov et al., 2023) has shown promising results dealing with value misalignment issues (Bradley Knox & Stone, 2008; MacGlashan et al., 2017; Christiano et al., 2017; Ibarz et al., 2018; Goyal et al., 2022), unlocking new possibilities for chain of thoughts (Wei et al., 2022b), tree of thoughts (Yao et al., 2023a), and graph of thoughts interactions (Besta et al., 2023). However, recent research also highlights the limitations of LLMs in functional linguistic competence despite their proficiency in formal linguistic competence (Mahowald et al., 2023). Whereas formal linguistic competence encompasses the ability to understand and generate language, functional linguistic competence pertains to the application of language in real-world contexts, such as conveying sensory input or recalling information from memory. Examples of functional linguistic competence include implicatures (Ruis et al., 2022) and contextual language comprehension beyond the statistical manifestation of data distributions (Bransford & Johnson, 1972; Mikolov et al., 2013b). Consequently, operating LLMs through a purely inference-based approach confines their capabilities within their provided context window, severely limiting their horizon. This results in deficiencies for situational modeling, non-adaptability through contextual changes, and short-term problem-solving, amongst other capabilities. However, simply increasing the context length may not yield greater capabilities, as demonstrated by the observed U-shaped performance curve (Liu et al., 2023) where LLMs excel when utilizing information at the beginning or end of the input context, but struggle with information located in the middle, especially as context increases. These challenges are actively being researched, with novel approaches such as Hyena (Poli et al., 2023), RWKV (Bo, 2021), GateLoop (Katsch, 2023), Mamba (Gu & Dao, 2023) and xLSTM (Beck et al., 2024) surfacing. Meanwhile, the re-emergence of interest in retrieval-augmented generative approaches (Li et al., 2022a) offers an alternative by circumventing the autoregressive nature of the widely-utilized Transformer architecture (Vaswani et al., 2017), enabling context enrichment with lateral information.

In-Context Learning Recently, several in-context learning methodologies evolved to enable tool usage through LLMs (Schick et al., 2023), or refine the generative outcome of LLMs (Yang et al., 2023). This includes chain-of-thought (CoT) prompting, a method that conditions the model to reveal its step-by-step reasoning process (Wei et al., 2022b; Singhal et al., 2023). CoT prompting breaks down complex tasks into simpler, sequential steps, and helps with interpreting LLM’s output. Self-generated CoT, where models are encouraged to generate their own reasoning chains based on training examples, surpasses even expertly crafted CoT (Fernando et al., 2023). This observation echoes other reports that GPT-4 has an emergent self-improving capability through introspection, such as self-verification (Weng et al., 2023) or self-consistency (Wang et al., 2023b). Tree of Thoughts (ToT) enables LLMs to solve complex problems by exploring multiple reasoning paths through a search tree of coherent text units, demonstrating significant problem-solving enhancements in tasks requiring strategic planning and search (Yao et al., 2023a). Ensemble techniques further enhance the robustness and accuracy of model predictions by combining several strategies to establish a consensus (Nori et al., 2023).

3 PROBLEM DEFINITION

Conventional approaches employing foundation models, such as LLMs, are predominantly confined to single-step or few-step executions and primarily reliant on hand-crafted prompt instructions, often referred to as in-context learning. This restricted scope limits the utilization of different modalities, lacks verification, and exhibits limited tool proficiency. We posit that the use of NeSy engines as core computation units, realized through logic-based methodologies coupled with sub-symbolic foundation models, offers a more general, robust, and verifiable perspective. This approach has several advantages. Firstly, it enables the integration of pre-existing solutions (e.g. various classical algorithms), offloading computational complexity and bridging different modalities. Secondly, it allows sub-symbolic components to focus on decision-making (e.g. selecting the respective tool based on in-context classification). Thirdly, it provides

an *interpretable language-based control layer* for explainable, autonomous systems. In the following section, we elaborate on the key design principles underlying SymbolicAI and how we guide the generative processes of NeSy engines. For further technical details, see Appendix Section 5.

4 DESIGN PRINCIPLES

Symbols and Expressions As posited by Newell & Simon (1976), symbols are elemental carriers of meaning within a computational context³. These symbols define physical patterns capable of composing complex structures, and are central to the design and interpretation of logic and knowledge representations (Augusto, 2022). We define a symbol as the set $\mathcal{S} = \bigcup_{n \geq 0} \mathbb{L}^n$ formed by concatenating characters from a finite character set $\mathbb{L}$, i.e. the vocabulary in an LLM setting, and with n representing the sequence length of the string. Thus, let the set of all possible symbols be defined as Σ and that $\mathcal{S} \in \Sigma$. We further introduce an operation $\oplus$ that enables us to create expressions on any number of symbols from Σ , and when evaluated returns a new symbol in Σ . For any subset $\{\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_m\} \subseteq \Sigma$, an expression is defined as $\omega : \bigoplus_{i=1}^m \mathcal{S}_i \rightarrow \mathcal{S}'$ from the set of all possible expressions $\omega \in \Omega$, where $\mathcal{S}' \in \Sigma$, and $\oplus$ represents the placeholder operation of combining and transforming the symbols according to specific rules for m number of symbols. Such a specific rule for $\oplus$ can define an arithmetic expression $\oplus := +$ where two symbols are added, i.e. $\omega := "1" + "two"$ which results in a new symbol "3" or "three". Thus, SymbolicAI is based on the concept that symbols, and the expressions they form, are reflections of the information inherent in a NeSy system, and serve as surrogate for the interaction between the NeSy system and the problem space. Moreover, we argue that *real patterns* (Dennett, 1991), recurring and identifiable structures that coherently and reliably emerge in the data beyond mere randomness or noise, can be effectively realized through symbols.

Furthermore, we utilize language as a tool for mapping complex concepts, leveraging its inherent semantics and abstractions to describe states and properties of a problem at hand. These mappings are universal, e.g. they may be utilized to define scene descriptions, long-horizon planning, acoustic properties, emotional states, physical conditions, etc. Therefore, language serves as a comprehensive, yet abstract framework to encapsulate meanings, and refer to it as the *convex hull of the knowledge of our society*. Subsequently, it is common to attribute existing physical objects with abstract concepts, as exemplified by our natural tendency to link tangible objects to colors and emotions, such as blending the color "red" with "heart", "warm", and "passion". This approach also anchors our work in the field of formal language theory, as we require a structured method to construct mappings from the world to language. Consequently, we use formal language structures, such as grammars, to systematically define our language-centric approach to problem-solving and the associated translation of real-world complexities into linguistic terms.

Formal Languages In formal language theory and linguistics, languages are structured following the Chomsky hierarchy, which classifies languages by the complexity of their grammatical structure (Chomsky, 1956). This hierarchy defines four types of grammars (Type-3 to Type-0) and separates formal languages by their grammatical complexity. A grammar in this context consists of terminal and non-terminal symbols, production rules, and a designated *start symbol*, enabling the generation of valid strings within a language.

We define a NeSy engine as a mapping $\mathcal{V}_{\mathcal{S}^*} : \Omega \times N \times \mathcal{T} \rightarrow \Sigma$, where $N \subset \Sigma$ is a set of non-terminal symbols, $\mathcal{T} \subset \Sigma$ is a set of terminal symbols and $N \cap \mathcal{T} = \emptyset$, and $\mathcal{S}^* \in \Sigma$ is a starting symbol. We further formalize a grammar $G = (N, \mathcal{T}, P, \mathcal{S}^*)$ with production rules defined as a $P := \mathcal{V}_{\mathcal{S}^*}(\omega, N, \mathcal{T})$. This grammar describes the generation of symbols through expressions ω . For simplicity, we will drop the subscript of $\mathcal{V}_{\mathcal{S}^*}$ and use it as $\mathcal{V}$. We identify LLMs as promising candidates for functioning as part of NeSy engines. In SymbolicAI, a symbol $\mathcal{S}$ is augmented with conditional instructions and types derived from DSLs, custom defined or not (e.g. HTML, SQL, etc.), tailored for directing the LLMs. The key advantage of LLMs over previous systems lies in their ability to generalize across formal languages (Wang et al., 2023a) and knowledge systems. Although there is currently no universal consensus regarding the precise classification of natural language within the Chomsky hierarchy, our approach can be understood as employing a *situation-specific*, context-sensitive grammar, which enables the processing of instructions and analogies with a nuanced understanding of language. The intersection between formal and natural languages becomes evident when considering how language patterns, through prompts like "You are a helpful assistant...", elicit structured responses, indicating a potential underlying formal mechanism at play. This observation underlines the utility of such a grammar in our framework, where it serves as an explicit schema guiding the structure of examples for in-context learning. For instance, equating "3.1415..." with " π " or "August 4, 1961" with "1961-08-04" in a given context demonstrates context-dependent interpretation of symbols. Such a system doesn't rigidly adhere to standard grammatical rules but instead adjusts and interprets based on the context, effectively creating a situation-specific gram-

³ Our framework's name is derived from the foundational work of Newell and Simon.

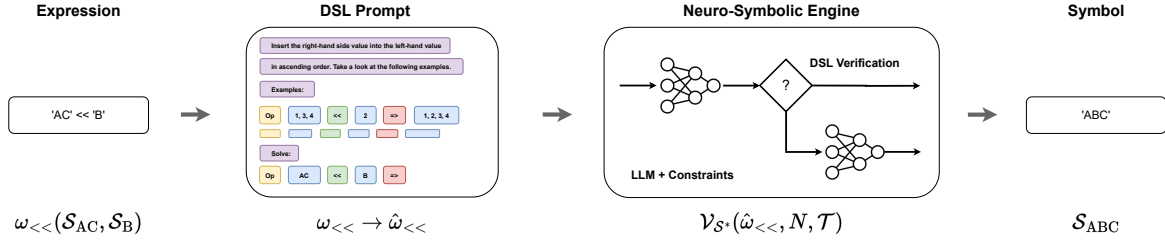


Figure 2: Illustration for NeSy pipeline, showcasing conceptual usage of in-context learning methodologies, domain-specific language (DSL) structures, and the expression evaluations through a NeSy engine based on an LLM and constraint verification. The expression showcases the sorted insert operator $\ll$ and how the information of the symbol B is included in the symbol AC. The violet placeholder in the *DSL Prompt* represents an instruction, such as “Insert the right-hand side value into the left-hand value in ascending order. Take a look at the following examples.” The positions below represent task-specific few-shot examples. The DSL Prompt receives the expression $\omega_{\ll}$ and maps it to $\hat{\omega}_{\ll}$ that can be processed by the LLM-based NeSy function $\mathcal{V}_{S^*}$ and outputs a new symbol.

mar, capable of forming *Domain-Invariant Associations* through in-context learning. We further address this in a later paragraph.

Function Composition In SymbolicAI, we use function composition to construct complex hierarchies and behaviors from fundamental elements. Therefore, our framework enables modeling of interconnected processes, where the output of one function is used as input for another, thus creating a sequence of operations. Through function composition, we construct computational graphs, in which intermediate symbols represent the nodes or states within these graphs. Formally, function composition is denoted by $\circ$, where combining functions f and g yields a new function $h = g \circ f$, defined as $h(x) = g(f(x))$. For functions $f : X \rightarrow Y$ and $g : Y \rightarrow Z$, their composition results in a function mapping elements from domain X to codomain Z through $g(f(x))$. Although traditionally the codomain of the inner function f aligns with the domain of the outer function g , SymbolicAI relaxes this constraint by allowing for any subset relationship between these domains and codomains, which is particularly beneficial for in-context learning. When using LLMs for NeSy production rules $\mathcal{V}$, we can derive a multi-step generative process by composing a computational graph as a sequence of zero- and few-shot function compositions:

$$\mathcal{V}(\omega_j, N, \mathcal{T}) = \mathcal{V}(\omega_{j-1}, \cdot) \circ \mathcal{V}(\omega_{j-2}, \cdot) \circ \cdots \circ \mathcal{V}(\omega_0, \cdot), \quad (1)$$

where ω_0 is the initial instruction and j defines the index variable for a multi-step generative process. By leveraging functional in-context learning, where zero- and few-shot examples act as dynamic elements of the function’s domain, SymbolicAI has the ability to interpret and respond to diverse input contexts. For instance, a function can classify a user request and select an appropriate interface (e.g. WolframAlpha) to process the request. The output modality may even vary based on the respective engine. This enables SymbolicAI to handle operations over multi-modal data that connects multi-step generative processes and establishes function composition as a central tenet in bridging multiple modalities and coordinating a variety of tasks.

Domain-Invariant Associations In-context learning enabled LLMs to become versatile task solvers by interpolating within the training distribution, to the extent that even potentially unseen tasks are addressable (Brown et al., 2020). We attribute this to associations formed within the input space and the capacity of Transformer architectures for defining domain-invariant feature sub-spaces. This phenomenon has parallels with few-shot learning approaches such as SubGD (Gauch et al., 2022), a method based on identifying and utilizing a low-dimensional subspace, learned from various tasks that effectively regularize the learning process. Since LLMs have been trained on different domains and tasks, which also include formulations of mathematical expressions, we posit that specific tokens, such as the equality sign, can be leveraged to associate meanings between different symbolic objects. Unlike domain-invariant representations that create invariant features across different learning tasks, our approach leverages the in-context generalization capability of LLMs to construct invariant symbolic associations that aim to preserve, manipulate and propagate situational context. We can use these properties to build operations that apply transformations on objects that are substitutes to the semantically aligned few-shot learning examples.

5 SYMBOLICAI FRAMEWORK

In this section, we discuss the specifics of the proposed SymbolicAI framework. For more details about the framework structure, see Appendix Section C. For installation and usage of our framework, see Appendix Section D. For more technical details and code snippets, see Appendix Section E.

Types and Representations Analogous to the type `object` in Python, the base type of SymbolicAI is a symbol represented by the base type `Symbol`. All other subtypes, such as `Expression`, represent their mathematical namesake and can be evaluated and simplified. These subtypes inherit from `Symbol` the base attributes, primitive operators, and helper methods.

Although SymbolicAI uses a language-centric design, modeling and manipulating every interaction into symbolic representations is not inherently efficient. Therefore, we establish mappings between symbolic and sub-symbolic representations for sensory inputs and non-discrete elements. Such mappings are typically realized through function approximation. This allows us to map between *modality*-to-language and language-to-*modality* use cases. Here, *modality* serves as a placeholder for various types such as text, image, video, audio, motion, etc. In turn, each `Symbol` object contains valued and vector-valued representations, obtained through `value` and `embedding` attributes. The latter represents a symbol’s current value, akin to embedding text and storing it as a PyTorch tensor (Paszke et al., 2019) or NumPy array (Harris et al., 2020). While for an LLM, the numerical tensors may lack inherent meaning, vector-valued representations play an important role when 1) composite symbols are combined into more complex expressions, and 2) these embedded tensors are updated through gradient-based optimization.

To enable the processing of symbols by LLMs, we assume that each `Symbol` object implements Python’s native string functionality, where the `__str__` method returns an interpretable string representation. Therefore, we can assert that any Python object is parsable by an LLM, however, the user must ensure a meaningful representation. For more details, see Appendix Section E.

Polymorphic Context Polymorphism is a central concept in programming language theory and prominently featured in SymbolicAI. Polymorphism refers to the ability of different objects to be accessed through the same interface, or of a single identifier to represent different types based on the context of execution. Providing a single interface for entities of different types allows operations to be performed in ways specific to their derived types. We therefore designed the `Symbol` object to contain a global context, which is composed of static and dynamic context parts, and enables this polymorphic behavior. The static context is class dependent and defined at design time. The dynamic context is runtime adaptable and can be changed to adhere to runtime specific logic and changes. Moreover, `Symbol` associated operations are resolved following polymorphic design before being evaluated by the NeSy engine. SymbolicAI’s engine implementation contains a `prepare` method to resolve and compile the engine specific representation by evaluating the `Symbol`-specific operations and context. For an example on polymorphic context see part a) in Figure 3.

Operators and Methods In SymbolicAI, operators are overloaded to facilitate transformations of `Symbol` objects. These operator primitives employ dynamic casting to ensure type compatibility. Consequently, `Symbol` objects can be easily manipulated through type specific attributions or symbolically evaluated by the NeSy engine. For example, a central operation for boolean logic is measuring equality between symbols. To evaluate the equality of symbols, we primarily adhere to the type specific implementation, because we prioritize strict comparisons over probabilistic evaluations. If the evaluation was unsuccessful, we then consider semantic equality through the NeSy engine. SymbolicAI leverages decorators for composing operators and custom class methods. For more details, see Appendix Section C.

Upon invoking an operator or method, the respective primitive function evaluates the symbol’s specific type and its respective attributes, and if necessary, resolves a nested decorated function that then uses the NeSy engine for evaluation. Should the evaluation fail, a predefined fallback implementation executes. Absent a fallback, or if both evaluations fail, an error state is raised. The processing of an operator or custom method involves a pipeline consisting of pre- and post-processing steps, as well as constraint enforcement. Constraints cover aspects like return types, value ranges, and structural integrity (e.g. JSON formatting through grammar-based verification). In Figure 3 b) we give an overview of the entire prompt composition based on the user input, the `Symbol` object structure, and in part c) the `Symbol` evaluation pipeline.

Self-Referential Structures SymbolicAI augments the generative process by enabling systems to introspect and modify their behavior dynamically. We leverage LLMs to execute tasks based on both natural and formal language instructions, adhering to the specified user objectives and with innate self-referential structures. We derive subtypes from `Expression` and enclose their functionalities in task-specific components, which we then expose again through

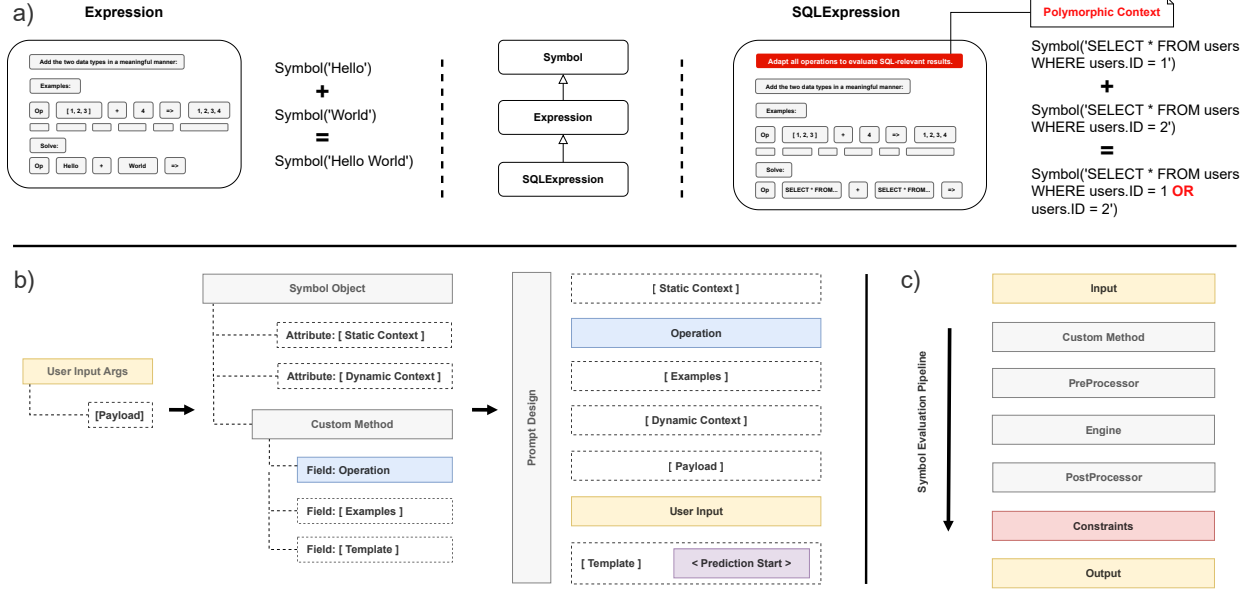


Figure 3: **a)** Illustration of polymorphic context on the example of a `SQLExpression` type for the `add`-operator. Without a polymorphic context a regular `Expression` evaluation concatenates two `Symbol` objects together. The polymorphic context in `SQLExpression` overwrites the base behavior such that two added SQL-expressions get semantically combined, not concatenated. **b)** Illustration of the translation of a `Symbol` object to a prompt statement to be processed by an LLM in the NeSy engine. The `User Input Args` can be attached with a `Payload` from previous executions and gets applied to the `Custom Method`. The user input with the polymorphic context of the `Symbol` Object attributes (`Static Context` and `Dynamic Context`) are translated to a prompt statement according to the schema of the `Prompt Design`. The fields `Operation`, `Examples` and `Template` mark operation description, DSL-based prompt examples and template structures respectively. These translations are processed according to `PreProcessor` and engine-specific formatting. **c)** Illustrates the evaluation pipeline from user input to output, with multiple translation processes before and after the `Engine` invocation. The `Input` gets passed to the `Custom Method` and reformatted according to a `PreProcessor` to adhere to DSL-specific structure. The engine then takes the output of the `PreProcessor` and composes the final prompt according to the engine-specific `Prompt Design` and resolves polymorphic context and auxiliary fields. The output of the `Engine` then can be restructured by a `PostProcessor` to match DSL-requirements of the desired `Output` and gets applied `Constraints` to verify the outcome.

templating and the model-driven design of the NeSy engine. This design choice allows a system to create and utilize its own sub-process definitions, analogous to concepts discussed in Schmidhuber (2007; 2009). Concretely, we utilize generalization properties of LLMs to interpret and formulate a set of operations that incorporate *self-instructions* (Wang et al., 2022). Consequently, the operations hold the flexibility to adapt to the context, and derive sub-processes that self-instruct LLMs to engage in situational modeling and context-sensitive problem-solving. Ultimately, this enables the construction of hierarchical computational graphs for self-referential *meta-reasoning* systems without the need to explicitly training a meta-learner (Kirsch & Schmidhuber, 2022). In Figure 4 we illustrate a step-wise evaluation of a contextual computational graph, in which the NeSy engine is processing conditioned on the current execution context and producing a next symbol prediction.

6 PERFORMANCE MEASURE

One of the challenges when creating multi-step generative processes with LLMs as part of NeSy engines relies on model evaluation and handling irrelevant predictions. The naïve assessment that measures only task succession would score all models to zero and render them as unusable. Even if models follow instructions and produce parts of the expected solution, we regularly observe that they — especially open-source models — append a continuation of task irrelevant predictions. Such predictions result in failure modes when applying conditions and validations, and halt any multi-step procedure. Our solution is an evaluation protocol that refines the performance measurement, allowing for more nuanced diagnostics and the possibility of continuing the evaluation despite intermediate failures. To derive our

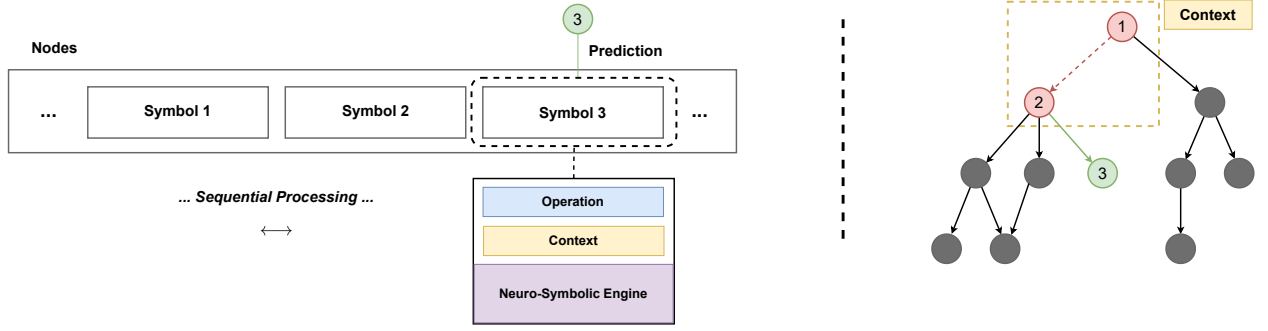


Figure 4: We showcase a multi-step hierarchical computational graph, with each node in the graph represented by a symbol. The edges are relations between symbols. The left-hand side illustrates how a new node (Symbol 3) is obtained by evaluating an operation with its respective context on a NeSy engine. The right-hand side illustrates the context information window (yellow rectangle) and relationship of the resulting graph with its respective nodes.

quality measure, we borrow ideas from the utilization of the Fréchet distance for generative processes (Heusel et al., 2017).

We generate trajectories through a NeSy sequential process that creates a trajectory of distributions $\mathbb{P}$ over multiple iterations of generative nodes. Each node in the process can be aligned to a reference distribution, which marks the desired behavior. To quantify the validity of the generated trajectories, we measure the total distance between the generated and reference data distribution along the path trajectory. We therefore adopt a cumulative measure capable of taking into account the entire generative trajectory. In theory, this process would entail calculating the path integral over the latent space representations for models, cumulating the Fréchet distances (Dowson & Landau, 1982) traversed along these trajectories:

$$\mathcal{D}(\mathbb{P}_{\text{gen}}, \mathbb{P}_{\text{ref}}) = \int_{t_0}^{t_f} d(\mathcal{N}(m_t, C_t), \mathcal{N}(m_{w,t}, C_{w,t})) dt \quad (2)$$

where $\mathcal{D}(\mathbb{P}_{\text{gen}}, \mathbb{P}_{\text{ref}})$ denotes the integral of the Fréchet distances between two data distributions along the generative path trajectory from an initial time t_0 to a final time t_f , $d(\mathcal{N}(m_t, C_t), \mathcal{N}(m_{w,t}, C_{w,t}))$ is the Fréchet distance calculated at each time t between the generated multivariate normal data distribution with mean m_t and covariance C_t , and the reference multivariate normal data distribution with mean $m_{w,t}$ and covariance $C_{w,t}$. The resulting measure follows properties of normal distributions and is consistent with increasing disturbances.

However, this approach is computationally intractable for large-scale problems, and requires access to latent representations, which — especially in the context of LLMs — is not always given. For computational feasibility, we introduce an approximation that measures the embedding distances over the path trajectories through an auxiliary embedding model, based on prior work on distribution regression (Szabó et al., 2016). The embedding model maps the symbolic representations into a RKHS, such that we can apply a kernel mean embedding function to measure their respective distances (You et al., 2019; Dinu et al., 2023). We assess the distance through the mean embeddings w.r.t. to a kernel function $K(\cdot, \cdot)$ of the samples $\mathbf{e}_x^t \sim \nu_{\text{gen}}^t \in \mathbb{P}_{\text{gen}}$ and $\mathbf{e}_y^t \sim \nu_{\text{ref}}^t \in \mathbb{P}_{\text{ref}}$ produced by the generated data distribution and a reference data distribution respectively. We denote by $\mu_{\mathbf{e}_x^t}, \mu_{\mathbf{e}_y^t}$ the mean embeddings associated to the respective samples, i.e. $\mu_{\mathbf{e}_x^t}(z) = \frac{1}{n} \sum_{i=1}^n K(x_i^t, z)$ in case $\mathbf{e}_x^t = (x_i^t)_{i=1}^n$ is a sample of size n of the respective mean embeddings. To compute the similarity between the embeddings of the generated and reference distributions, we evaluate the associated maximum mean discrepancy $\text{MMD}^2(\mu_{\mathbf{e}_x^t}, \mu_{\mathbf{e}_y^t})$ (Gretton et al., 2012) and then, as before for the Fréchet distances, we integrate over t :

$$\tilde{\mathcal{D}}(\mathbb{P}_{\text{gen}}, \mathbb{P}_{\text{ref}}) = \int_{t_0}^{t_f} \text{MMD}^2(\mu_{\mathbf{e}_x^t}, \mu_{\mathbf{e}_y^t}) dt. \quad (3)$$

In empirical evaluations, however, we care about normalized values for ease of interpretation. We therefore analyze the properties of the MMD and derive a similarity score, which follows the same statistical principles as the MMD, and is bound between $[0, 1]$. We concluded that we can utilize only the MMD cross terms to evaluate the similarities. See Appendix Section B for more details. For our comparisons as referenced in Figure 6 we therefore denote the similarities rather than distances. We then come to the following formulation and refer to our empirical measure as the "Vector Embedding for Relational Trajectory Evaluation through Cross-similarity", or *VERTEX* score for short:

$$s(\mathbb{P}_{\text{gen}}, \mathbb{P}_{\text{ref}}) := \int_{t_0}^{t_f} \left[\min(\max(0, \frac{1}{z} \widetilde{\text{MMD}}^2(\mu_{\mathbf{e}_x^t}, \mu_{\mathbf{e}_y^t}) - z_{\text{rand}}), 1) \right] dt. \quad (4)$$

We approximate the integral across time steps through Monte Carlo approximation. The introduced normalization constants denote the similarities to a random sequence z_{rand} , which functions as a baseline subtraction to recenter our results, and a given reference score to rescale w.r.t. to scores obtained from comparing related solutions z . Min-max scaling ensures the final measure is bounded between $[0, 1]$. This process reflects properties such as Hölder continuity that bounds the kernel function within certain limits. To compute the embeddings, we utilize the embedding model `all-mpnet-base-v2` (Song et al., 2020), due to its widespread availability, and its balance between speed and quality. As a similarity measure, we select a Gaussian kernel following our derivation from the Appendix Section B. In our implementations, we also explore other kernels, including preliminary experiments with cosine similarity. We also note that one can integrate Bernoulli distributed trials into our score, with 0 values representing failure modes and values of 1 being successes. Furthermore, if we relax our definition, we can integrate other similarity measures which are bound between $[0, 1]$, which then reflect on domain-specific attributions, i.e. including a similarity measure tailored towards capturing the nuances between two sub-structures of abstract syntax tree.

7 EVALUATION

We introduce a benchmark that evaluates multi-step generative processes as complex workflows. Our benchmark consists of five different evaluation categories, and uses the VERTEX score to measure the capabilities of an LLM to solve tasks from each category. The five categories of our benchmark are divided into three basic evaluations and two advanced categories that combine different basic capabilities. The three basic categories are (i) **associative prediction** which measures a models proficiency in understanding associations between symbols, (ii) **multi-modal binding** where we evaluate the capability to employ tools and operate on different modalities, and (iii) **program synthesis** for measuring a models proficiency in generating consistent code and executing subroutines. The two advanced benchmark categories are (iv) **logic**, for evaluating logic-based components and (v) **computational graphs** where complex workflows need to be processed, evaluating all aforementioned capabilities. For our evaluation we focus on the GPT family (Brown et al., 2020) of models, specifically GPT-3.5 Turbo (revision 1106) and GPT-4 Turbo (revision 1106) as they are the most proficient models to date; Gemini-Pro (Google, 2023) as the best performing model available through API from Google; LLaMA2-Chat 13B (Touvron et al., 2023), LLaMA3-Chat 8B and LLaMA3-Chat 70B from Meta represent open-source LLMs. Finally, Mistral 7B (Jiang et al., 2023) and Zephyr 7B (Tunstall et al., 2023) serve as baselines for revised and fine-tuned open-source models. The open-source models Mistral, Zephyr, and smaller LLaMA variants are estimated to have roughly equivalent parameter counts compared to GPT-3.5 Turbo and Gemini-Pro. All our experiments require a context size smaller or equal to 4096 to enable the comparisons among the in-context capabilities across model architectures. For the LLaMA models, we utilize the *chat* versions since they are specifically tuned to follow instructions.

Associative Prediction We evaluate a model’s proficiency to follow simple and complex instructions and associations with zero- and few-shot examples. Therefore, we evaluate the proficiency in applying our operators between `Symbol` types. We defined a total of 15 tasks involving in-context associations between two `Symbol` instances. SymbolicAI’s overloaded operators rely on predefined pseudo-grammars, as described in Section 4, that augment the operators with few-shot examples. For instance, the overloaded operator `+` utilized between two `Symbol` instances provides few-shot examples how to resolve additions with various data types. Consequently, we can now test if the models can solve the addition between `Symbol("two hundred and thirty four")` and `Symbol(7000)`. See Appendix Section F.1 for more details.

Multi-modal Binding We perform transformations between multiple modalities through language-based representations. Therefore, we need to evaluate the model’s proficiency in tool utilization, classification and routing of requests to relevant modules. We define a multi-modal `Expression` to detect the category of a task based on its content and to forward the task to the appropriate tool. The expression creates interfaces to tools like WolframAlpha for mathematical expressions, Selenium for website content scraping, SerpApi for search queries, and APILayer for optical character recognition. Each of the five tests aims to evaluate the appropriate handling of a specific type of input by the multi-modal `Expression` type, such as processing a website URL for scraping, interpreting a search engine query, testing if two vectors are linearly independent, comparing large numbers, and extracting text from an image. See Appendix Section F.2 for more details.

Program Synthesis We evaluate executable code with and without concepts from retrieval augmented generation, model-driven development, and experiment with self-generated instructions by creating self-referential expressions. We designed three separate tests related to program synthesis, where each task assesses the ability of the models to generate and execute code based on natural language instructions or provided templates:

- 1) The first task involves reading a LaTeX table template and data, then generating a function to populate the table with the given data.
- 2) The second task tests the automatic code generation for API calls by fetching data from a specified URL and extracting specific information from the retrieved content.
- 3) The third task evaluates the ability to construct a custom `Expression` that processes a `Symbol` through a specific `Function` component from the `SymbolicAI` package.

Each of the three tests follows a similar pattern, where the generated code is scored based on its similarity to valid references and normalized with random samples. See Appendix Section F.3 for more details.

Logical Components To evaluation the capabilities for logical reasoning of models, we condition them to create a sequence of expressions as self-contained components, and refer to higher-order logic for their assessment. Based on the underlying *type theory* originating from [Whitehead & Russell \(1925–1927\)](#), we evaluate a models’ capability to resolve statements in the form of *there exists x such that x fulfills y* . Such quantifiers define the standard semantics of expressions, where their meaning is given by a semantic function. A semantic function maps a term from an abstract definition to a point in a domain, which is an interpretation of the term’s type and value. Therefore, these functions operate on types and values of expressions, and relations thereof. Subsequently, NeSy engines can formulate and evaluate at inference time logic-based instructions through Lisp, Prolog, or Mathematica ([McCarthy, 1959](#); [Colmerauer & Roussel, 1993](#); [Chen et al., 1993](#); [Inc., 2022](#)), or leverage solvers such as Z3 ([Moura & Bjørner, 2008](#)). Therefore, the result of a natural language statement when evaluated by a NeSy engine can be interpreted by any expert system which defines the corresponding semantic functions and process them either in a symbolic ([Feigenbaum et al., 1965](#); [Gamble et al., 1994](#)), differentiable ([Veličković & Blundell, 2021](#); [Ibarz et al., 2022](#)), or hybrid manner ([Kuncicky et al., 1991](#)).

We evaluate how proficient models are at interpreting custom DSLs and define expression statements. DSLs are designed to express logical relations and operations in a structured format, and supports human-readable and machine-interpretable formulations. The following example illustrates such relationships by translating a natural language statement into an expression statement, as follows:

Marvins has four paws and likes to meow when I pet its fur. Is Marvins a cat?

A DSL may enforce the usage of `HAS(·)`, `IS(·)`, etc. and may condition an LLM to produce the following expressions:

- `HasFourPaws( $x$ )`: x has four paws.
- `LikesToMeowWhenPetted( $x$ )`: x likes to meow when it is petted.
- `IsCat( $x$ )`: x is a cat.

These are then utilized to define the following logical expression:

$$\forall x (\text{HasFourPaws}(x) \wedge \text{LikesToMeowWhenPetted}(x) \Rightarrow \text{IsCat}(x)).$$

An automated theorem prover can now evaluate this statement for all values of x and assess the validity of the original query. Lastly, our evaluation uses symbolic mathematics to manipulate algebraic expressions. This involves defining symbols and performing operations like factorization, simplification, and algebraic manipulation. The symbols are placeholders for any value, enabling the definition of general expressions without specifying their values upfront.

We designed six tests to assess the logical capabilities of the candidate models and group them as follows. See Appendix Section F.4 for more details.

- 1) We utilize the Python library SymPy for symbolic mathematics to create the mathematical expression $ax + bx - cx - ay - by + cy + d$. The task for the model is then to factorize the expression and extract all unique symbols as a list.
- 2) Three tasks evaluate a models’ capability to resolve the logical operations AND, OR, and XOR. For instance, the test for logical AND combines the symbols `Symbol("The horn only sounds on Sundays")` and `Symbol("I hear the horn")` and compares the answer against the human-generated references *"The horn only sounds on Sundays and I hear the horn."* and *"Since I hear the horn it is Sunday."* Since there is a large number of possible solutions, there is high variability in the solution space. Each model might prefer a different solution.

3) For another task we use a custom `Expression` that defines a DSL syntax and semantic structure. We use this `Expression` to extract higher-order logic expressions from a natural language statement, namely the puzzle ‘Who is Jay’s brother?’⁴, that preserves the original relationships.

4) For the final task, we again use the puzzle ‘Who is Jay’s brother?’ to evaluate a models’ capability for complex conversions. We use the Z3 theorem prover (Moura & Bjørner, 2008) to solve the ‘Who is Jay’s brother’ puzzle conditioned on the Z3 solvers’ solution to Einsteins’ famous puzzle ‘Who owns the fish?’. The task involves an indirect translation from natural language to executable code by the Z3 solver; the solution to Einstein’s puzzle acts as a form of self-contained “documentation” for how the Z3 solver should be utilized. The test constructs a template, which includes the task instructions, puzzle statement, and reference to the Einstein’s puzzle solution. The models are then asked to analyze the given problem and solution format and create a Python function with Z3 syntax that can solve the ‘Who is Jay’s brother?’ puzzle. The dynamically generated code is executed within the test environment utilizing Python’s `exec` function. We check the access to the Z3 solver and run the generated `solve_puzzle` function supposed to contain the logic to solve the puzzle. Once executed, the assembled Z3 logical clauses are processed by the solver, which verifies that the set of constraints is satisfiable. If so, the model generated by the solver is queried for the puzzle’s solution and scored using our VERTEX score.

Hierarchical Computational Graphs We evaluate the capabilities of models to orchestrate a multi-step generative process and evaluate a set of tasks. Models need to direct sub-processes and associate computational results from and to `Symbol` nodes, and maintain relationships between these nodes, which we refer to as a computational graph as shown in Figure 5. In a computational graph, the VERTEX score compares the results produced by a generative model at each node against samples obtained from a reference distribution, usually modeled by sampling from multiple valid references. We also account for randomness through predefined random samples for normalizing the result. Our reference to *hierarchical* computational graphs stems from the fact that we operate on multiple levels. On a higher level of abstraction we are able to perform planning, sub-task scheduling, and define operational instructions. On a lower level of abstraction, we execute these plans based on the defined instructions and data, which can also span generative processes that produce new information.

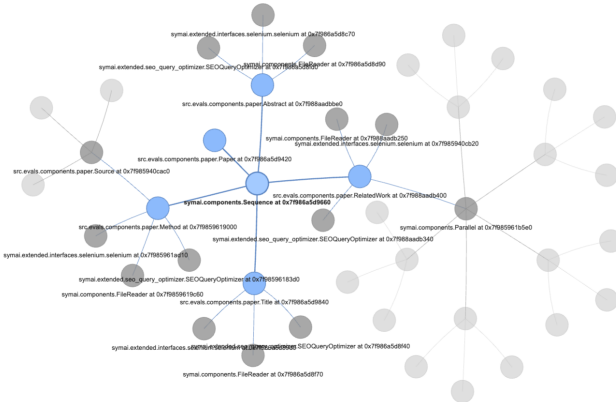


Figure 5: We illustrate the hierarchical computational graph for the Paper expression. Each node represents an instance of an expression with distinct properties and behaviors, such as file sourcing, generative process, tool utilization, or transformation operation. The edges denote the reference relationships between expressions and indicate the flow of information. The blue highlighted nodes mark the main sequence nodes of expressions utilized to create parts of the paper draft, such as Method section, RelatedWork section, Abstract section, and so on. Each generative node is used for evaluating the VERTEX score. None-generative nodes such as search engine results are not evaluated, and we assume to obtain ground-truth values.

Given that the field is currently at an early stage in developing even sequential schedulers for LLM-based planning systems, our evaluations will be confined to sequential execution only. We introduce two tests designed to evaluate multi-step generative processes:

- 1) We simulate and evaluate the process of writing a research paper draft based on a predefined hierarchical computational graph that focuses on the content output of the computational graph rather than planning and scheduling functionality. See Appendix Section F.5 for more details.
- 2) We test the VERTEX Protocol as defined in Algorithm 1, which represents our general method for evaluating multi-step generative processes. We create a self-contained test scenario to illustrate an end-to-end evaluation and as a go-to reference for how our protocol can be deployed in a realistic environment. Our evaluation protocol is not only designed to analyze and score a series of instructions, but also to provide a structured basis for recording these

⁴ Bob has two sons, John and Jay. Jay has one brother and father. The father has two sons. Jay’s brother has a brother and a father. Who is Jay’s brother?

processes. Furthermore, we note that our evaluation protocol is generally formulated, which allows the application of non-sequential planning and scheduling.

Algorithm 1 VERTEX Protocol

Require: NeSy engine $\mathcal{V}$ as an LLM, embedding engine $\mathcal{E} : \Sigma \rightarrow \mathcal{H} \subset \mathbb{R}^d$, symbols $\{x_0, x^*, y^*\} \subset \Sigma$, with x_0 as the initial instruction, x^* as the payload resulted from executing $\mathcal{V}$, y^* as the reference, and $*$ acting as a placeholder for $\mathcal{P}, \mathcal{T}, \mathcal{C}$, capabilities $\mathcal{C} = \{\mathcal{F}_1, \mathcal{F}_2, \mathcal{F}_3, \dots\}$, where each $\mathcal{F}_i$ represents a specific functional role within the system, plan $\mathcal{P} \subset \Sigma$, task $\mathcal{T} \in \mathcal{P}$, memory buffer $\mathcal{M} \subset \Sigma$, a scoring function $\tilde{s} : \mathcal{H} \times \mathcal{H} \rightarrow [0, 1]$, a scheduler $\mathcal{Q}$, an aggregator $\mathcal{A}$, and score variables $\{s\} \in [0, 1]$.

Method:

```

1:  $\mathcal{V}, \mathcal{E}, \mathcal{Q}, \mathcal{C}, y^{\mathcal{P}} \leftarrow \text{INIT}(\cdot)$                                 ▷ Initialize engines, scheduler, capabilities, expected plan.
2:  $\mathcal{M} \leftarrow \emptyset, \mathcal{A} \leftarrow \emptyset$                                 ▷ Initialize memory buffer and aggregator.
3:  $x^{\mathcal{P}} \leftarrow \text{GENERATEPLAN}(x_0, \mathcal{V})$                                 ▷  $\mathcal{V}$  generates plan based on initial instruction.
4:  $\text{EVALUATE}(x^{\mathcal{P}}, y^{\mathcal{P}}, \mathcal{E}, \mathcal{A}, \tilde{s})$                                 ▷ Embed, score, and aggregate plan similarity.
5:  $\mathcal{P}, \mathcal{M} \leftarrow \text{UNFOLDPLAN}(y^{\mathcal{P}}, \mathcal{M}, \mathcal{Q})$                         ▷  $\mathcal{Q}$  unfolds plan into actionable tasks and updates progression.
6: while  $\mathcal{P} \neq \emptyset$  do                                            ▷ Run until list of tasks is exhausted.
7:    $\mathcal{T}, y^{\mathcal{C}}, y^{\mathcal{T}} \leftarrow \text{SELECT}(\mathcal{M}, \mathcal{V})$                     ▷  $\mathcal{V}$  selects next task based on task progression.
8:    $\mathcal{F}_i \leftarrow \text{IDENTIFY}(\mathcal{T}, \mathcal{C}, \mathcal{V})$                             ▷  $\mathcal{V}$  identifies task-related capability  $\mathcal{F}_i$ .
9:    $x^{\mathcal{C}}, x^{\mathcal{T}} \leftarrow \text{EXECUTE}(\mathcal{T}, \mathcal{F}_i, \mathcal{Q})$                 ▷  $\mathcal{Q}$  executes  $\mathcal{T}$  with capability  $\mathcal{F}_i$  and assign results  $x^{\mathcal{C}}, x^{\mathcal{T}}$ .
10:   $\text{EVALUATE}(x^{\mathcal{C}}, y^{\mathcal{C}}, x^{\mathcal{T}}, y^{\mathcal{T}}, \mathcal{E}, \mathcal{A}, \tilde{s})$                 ▷ Embed, score, and aggregate capability similarity.
11:   $\mathcal{P}, \mathcal{M} \leftarrow \text{UPDATE}(\mathcal{T}, \mathcal{P}, \mathcal{M}, \mathcal{Q})$                     ▷  $\mathcal{Q}$  updates plan and progression.
12: end while
13:  $s \leftarrow \text{FINALIZE}(\mathcal{A})$                                 ▷ Finalize aggregation of scores.
14: return  $s$                                                     ▷ Return aggregated score of plan execution.
```

Algorithm 1: This algorithm defines the pseudocode of our VERTEX protocol with our respective VERTEX score as a scoring criteria. We start by initializing the NeSy engine $\mathcal{V}$, the embedding engine $\mathcal{E}$, the scheduler $\mathcal{Q}$, and a set of capabilities $\mathcal{C}$. The initial instruction x_0 is utilized to generate a plan $x^{\mathcal{P}}$ through $\mathcal{V}$. The plan and its expected outcome $y^{\mathcal{P}}$ are embedded, and their similarity is scored according to our VERTEX score and aggregated. The plan is then unfolded into actionable tasks. Each task $\mathcal{T}$ is selected and executed with the appropriate capability $\mathcal{C}$, resulting in the capability and task results $x^{\mathcal{C}}, x^{\mathcal{T}}$, and expected outcomes $y^{\mathcal{C}}, y^{\mathcal{T}}$ updated in the memory buffer $\mathcal{M}$. The process continues, with each task's result being embedded, scored, and aggregated until the plan is complete. The final aggregated score s is returned, reflecting the overall effectiveness of the plan execution.

We start with a high-level workflow description which consists of a list of tasks and optionally their respective sub-tasks; we refer to this as the plan $\mathcal{P}$. To perform the experiment, we utilize an expected plan $y^{\mathcal{P}}$ which was handcrafted for this evaluation. The expected plan is a queue of predefined tasks (in a particular order) that the system should follow to achieve the goal. The goal statement defines the end objective that the workflow aims to accomplish. We also have a set of plans similar to the expected plan, which are trajectories in the solution space, as well as the plan $x^{\mathcal{P}}$ that the LLM generates utilizing the GENERATEPLAN call for a specific seed. We score the predicted plan against the expected plan and the trajectories, then we continue to the next phase in which we utilize the expected plan to execute the tasks. At each step, the LLM will receive in its context the goal, the tasks, the current progress, and a query asking for the next task to execute; we refer to this as the memory buffer $\mathcal{M}$. If the LLM is not able to predict the next task, it will return a failure, and the expected plan will be utilized to execute the next task. The LLM has access to a predefined set of capabilities $\mathcal{C}$, specifically WolframAlpha, SerpApi, Selenium, and the LLM itself, which also represents our self-referential structure. We keep executing tasks until the queue is exhausted, and at each step, we utilize the EVALUATE call to measure the performance of the LLM with our VERTEX score. The scheduler class $\mathcal{Q}$ oversees the execution of the test workflow. It takes the setup configuration and orchestrates the linear execution of tasks, utilizing the expected plan as a reference. It maintains a pool of tasks to be executed and updates progress as tasks are completed. The UNFOLDPLAN call is a method of the scheduler class $\mathcal{Q}$. The method calls itself recursively until there are no tasks left. The SELECT call is responsible for determining which task to execute next from a pool of remaining tasks. It utilizes the LLM through self-reflection (Shinn et al., 2023) to choose the most suitable next task based on a template that gets progressively updated in the memory buffer $\mathcal{M}$ by the UPDATE call. The IDENTIFY call uses self-reflection and similarity scoring to determine the best interface based on the task at hand, then passes the interface to the EXECUTE call to execute the task. Lastly, the test ends with the FINALIZE call, which provides an aggregated assessment of the model's ability to manage and execute the workflow.

In Figure 6 we conclude with our evaluation and compute the cumulative score for all described evaluation categories and in the next section we discuss how to interpret the results of our framework.

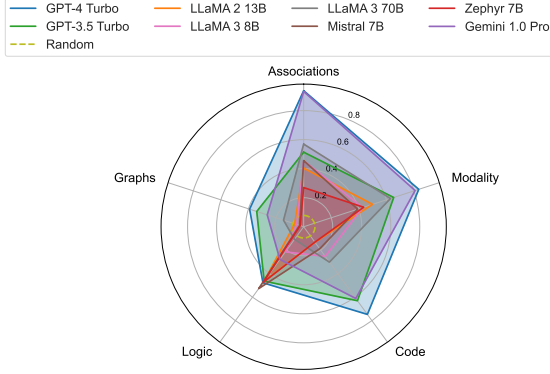


Figure 6: We evaluate GPT-4 Turbo, GPT-3.5 Turbo, Gemini-1.0 Pro, LLaMA2-Chat 13B, LLaMA3-Chat 8B, LLaMA3-Chat 70B, Mistral 7B and Zephyr 7B on five benchmark categories: 1) Associative Prediction (Associations) 2) Multi-modal Binding (Modality) 3) Program Synthesis (Code) 4) Functional Logic Components (Logic) and 5) Hierarchical Computational Graphs (Graphs). We denote the VERTEX scores for each category as a normalized value between 0 and 1, where higher values are better. The VERTEX score is measured according to a reference baseline and normalized by random sequences to exclude noise and similarities among references distributions to rescale solutions. The shown scores are an average over all tests per category and across 8 different seeds per test.

Benchmarks	GPT-4 Turbo	GPT-3.5 Turbo	Gemini 1.0 Pro	LLaMA 2 13B	LLaMA 3 8B	LLaMA 3 70B	Mistral 7B	Zephyr 7B	Random
Associations	0.94	0.51	0.93	0.40	0.46	0.57	0.46	0.27	0.08
Modality	0.83	0.65	0.81	0.50	0.43	0.63	0.39	0.43	0.07
Code	0.75	0.63	0.61	0.13	0.25	0.30	0.19	0.13	0.00
Logic	0.48	0.46	0.28	0.46	0.21	0.11	0.53	0.47	0.00
Graphs	0.39	0.34	0.26	0.06	0.05	0.15	0.03	0.03	0.00
Total	0.68	0.52	0.58	0.31	0.28	0.35	0.32	0.27	0.03

8 DISCUSSION

In this section, we address the evaluation results, auxiliary findings and limitations of SymbolicAI and the future directions we are focusing on. Some of the limitations stem from the inherent constraints of current technologies and dependencies on third-party systems. Additionally, the nuanced complexities of working with generative models presents further challenges.

8.1 RESULTS

In Figure 6 we show the VERTEX score for all five evaluation categories on 8 different state-of-the-art models. We show the aggregated results per category, meaning the average score among all tests averaged per category and average across 8 different seeds per test. The VERTEX score is normalized between 0 and 1, where higher values are better. Our score is non-linear due to its nature of using non-linear kernels, and captures semantic, ordinal and relative structures among the data samples. However, since our score is highly dependent on the quality of the underlying embedding model, it may omit to capture fine-grained syntactic differences such as ‘Hello’ vs ‘hello’.

In our experiments, we have noticed that for associative predictions and multi-modal bindings, GPT-4 Turbo is on par with Gemini-1.0 Pro. Furthermore, there is still a large gap between open-source contestants such as LLaMA 3 even with 70B parameters compared to the closed-source alternatives from OpenAI and Google. For the rest of the experiments, we see that GPT-4 almost always outperforms all other models, except for the functional logic components category. Here, we analyzed results and found that the larger models sometimes take shortcuts by automatically returning the solution and answering that the task instructions are too complex for such a straight-forward puzzle query. However, we would rather state in general that for logic-based, planning and scheduling tasks all models act unreliably, even if slight performance differences between the models are seen in the plot. We believe this is in part due to lack of training data specifically for workflows, planning and scheduling tasks, and to imprecision in generating reliably structured output formats, such as custom DSLs or other custom in-context instructed formats. This also stems from their instruction fine-tuning, since most models are chat-based models and offer verbose responses which need to be suppressed or post-processed.

We see similar performance between GPT-3.5 Turbo and LLaMA 3 70B except for the logical and graphs evaluations. We found that LLaMA 3 70B has a tendency to ask questions back if it does not understand the request instead of following the specified instructions provided. We assume this also stems from the chat-based instruction fine-tuning. Zephyr 7B and Mistral 7B have shown on par capabilities in functional logic components with larger models, however

fail in program synthesis and hierarchical computational graphs experiments. We observe that they perform well when resolving the overloaded logic operators such as OR, AND and XOR, and show decent performance for text generation, but fail to resolve more complex instructions.

8.2 LIMITATIONS

Framework Since the framework interfaces with many tools and API services, it requires a substantial engineering feat to integrate all available functionalities and keep the API-based services up-to-date. For us this means, that although we support a variety of tools and frameworks like Selenium, WolframAlpha, or Z3, we only scratch the surface of these tools. Moreover, the utilization of grammar-based constraints validations is still experimental and limited in functionality for specific formats such as JSON and HTML. Finally, we encounter also challenges related to engineering parallelization and multiprocessing of prompts, since the concurrent execution is non-trivial, especially with intricacies of Python process management.

Embedding Measure Our empirical measure is limited by the expressiveness of the embedding model and how well it captures the nuances in similarities between two representations. Furthermore, the obtained similarity scores are highly non-linear and difficult to interpret. For instance, two representations may address the same topic, such as the problem description and its respective solution, however, when measuring their similarity we obtain similarity scores of $\sim 70\%$. We normalize this by subtracting an inherent baseline and randomness effect, however, to ensure a more holistic and robust measurement we would need a significantly larger amount of baselines and experiments. Since we were very limited in the availability of development resources, and some presented models are only addressable through costly API walls. We are actively seeking sponsors to scale our solution and offer a more compelling benchmark suite in the future.

Model Capabilities An obvious limitation revolves around the fixed context window size of the underlying language models. Despite the expansion of the context window in newer models such as GPT-4, the finite context still restricts the amount of data that can be processed in a single pass. All information outside the context needs to be added through information retrieval approaches, which come with their own challenges and limitations (Gao et al., 2023). This leads to side effects, including hallucination, given the model does not contain the necessary information to answer the prompted instruction, which makes it difficult to maintain long-term statefulness for complex reasoning tasks and computational graphs.

Error Handling The complexity of error handling when evaluating complex expressions through function compositionality, especially between multiple modalities and different solvers, is another notable challenge. While SymbolicAI introduces mechanisms for error analysis and automated correction, these approaches are not infallible. They are often limited by the quality and expressiveness of the models, and the model’s capacity to understand deeply nested logical constructs. We also note that for our evaluations, we disabled any remedy protocol, such as truncating prompts or retry schema.

Generalization This research is also limited by current LLM’s capacity for reasoning and generalization. Although progress has been made, models are still prone to hallucinations and reasoning errors, especially when dealing with abstract, novel, or highly complex problem statements (Marcus, 2020). Furthermore, our framework’s reliance on the model’s ability to grasp the semantics of operations can be influenced by the training data and the model’s innate biases and conceptual understanding (Mahowald et al., 2023). We also point out that the initial development of SymbolicAI started with the GPT family of models, and we may encounter innate biases in prompt design and expressiveness when utilizing other reference models. However, we also point out that prompt engineering instruction-based statements is not a reliable direction for improvement. We instead advocate for enhancing the resilience of models through fault tolerance, focusing on their ability to better follow semantic instructions, not syntactic idiosyncrasies. Another concern is how to assess the disentanglement of evaluations of models on downstream tasks, to avoid testing on training samples, especially for closed-source solutions like GPT.

Interpretability and Transparency Finally, the issue of explainability and transparency in AI systems remains challenging. While SymbolicAI makes steps towards making computational processes more explicit and explainable through symbolic manipulations, understanding the internal logic and decision-making of LLMs remains an open problem. This can hinder trust and adoption in sensitive applications where interpretability of predictions is important.

8.3 FUTURE WORK

The goal for Algorithm 1 is to be utilized by an advanced learning agent. This agent, employing reinforcement learning methodologies (Ouyang et al., 2022; Li et al., 2023; Rafailov et al., 2023), could leverage our evaluation measure in existing benchmarks (Milani et al., 2020; Swazinna et al., 2022; Schweighofer et al., 2022) as a means to obtain reward signals to addresses a central problem in reinforcement learning, namely credit assignment (Sutton, 1984; Arjona-Medina et al., 2019; Holzleitner et al., 2020; Patil et al., 2020; Widrich et al., 2021; Dinu et al., 2022). Over time, it aims to develop the ability to autonomously generate its own plans, efficiently schedule tasks and subtasks, and carefully select the most suitable tools for each task. Our protocol lays the groundwork for this agent to learn and expand its base set of capabilities (Amaro et al., 2023), moving towards more sophisticated, self-referential orchestration of multi-step tasks. We’ve already noticed that research is shifting towards this type of methodology (Yuan et al., 2024). Furthermore, in Section 7 we’ve only considered a sequential scheduler. However, our objective is to ultimately assess a non-sequential task execution model, allowing for dynamic insertion and out-of-sequence task execution. In addition, we are interested into exploring similarities of our work with *Generative Flow Networks* (Bengio et al., 2021a;b; Lahlou et al., 2023). Lastly, we also discuss limitations in Appendix Section 8.2 with further opportunities for future improvements.

9 CONCLUSION

In this work, we introduced SymbolicAI, a framework that unifies generative models with an array of solvers, blending the strengths of symbolic and sub-symbolic AI paradigms within a cohesive NeSy framework. SymbolicAI equips researchers and practitioners with a comprehensive toolkit to develop contextualized and explainable NeSy AI systems capable of addressing diverse challenges effectively. We also introduce a quality measure and a benchmark for comparing and evaluating a wide range of computational tasks. SymbolicAI provides a basis for further research in advanced program synthesis, hierarchical computational graphs, the development of self-referential systems, and the integration of probabilistic methods with AI design for creating autonomous agents.

ACKNOWLEDGEMENT

The ELLIS Unit Linz, the LIT AI Lab, the Institute for Machine Learning, are supported by the Federal State Upper Austria. We thank the projects Medical Cognitive Computing Center (MC3), INCONTROL-RL (FFG-881064), PRIMAL (FFG-873979), S3AI (FFG-872172), DL for GranularFlow (FFG-871302), EPILEPSIA (FFG-892171), AIRI FG 9-N (FWF-36284, FWF-36235), AI4GreenHeatingGrids (FFG- 899943), INTEGRATE (FFG-892418), ELISE (H2020-ICT-2019-3 ID: 951847), Stars4Waters (HORIZON-CL6-2021-CLIMATE-01-01). We thank Audi.JKU Deep Learning Center, TGW LOGISTICS GROUP GMBH, Silicon Austria Labs (SAL), FILL Gesellschaft mbH, Anyline GmbH, Google, ZF Friedrichshafen AG, Robert Bosch GmbH, UCB Biopharma SRL, Merck Healthcare KGaA, Verbund AG, GLS (Univ. Waterloo), Software Competence Center Hagenberg GmbH, Borealis AG, TÜV Austria, Frauscher Sonsonic, TRUMPF, the NVIDIA Corporation and Atlas.

We extend our appreciation to Andreas Windisch and Clemens Wasner of AI Austria for their unwavering support. Their valuable feedback, connections, and facilitation of introductions within their expansive network have been instrumental to the progress of ExtensityAI.

Our gratitude also goes to Sergei Pereverzyev, whose enlightened guidance and thoughtful ideas have been a beacon for our research endeavors. Our thanks are equally extended to Gary Marcus, whose stimulating discussions sparked numerous innovative ideas incorporated into our framework.

We are equally grateful to Markus Hofmarcher, a friend and colleague whose informed counsel and stimulating discussions have significantly sharpened various facets of our study. Additionally, our thanks are due to Fabian Paischer and Kajetan Schweighofer, whose preliminary work and assistance have been of enormous benefit.

We are also grateful to our friends John Chong Min Tan and Tim Scarfe, whose communities have been a hub for exhilarating discussions. Their online presence and engagement have enriched the AI research landscape and broadened our perspectives.

Moreover, we wish to honor the memories of the cherished family members we lost in 2023. Their influence in our lives extended beyond personal bonds, and the principles they instilled in us continue to shape our journey. It is with great respect and affection that we acknowledge the indelible impact they have made, enabling us to persist in our scientific pursuits with determination and integrity.